FICHE DE SYNTAXE KOTLIN

L'essentiel pour développer des applications Android avec XML Views

Objectif : disposer d'un aide-mémoire simple pour lire, écrire et comprendre du code Kotlin dans Android Studio. Les exemples privilégient les bases du langage et les situations rencontrées dans les activités Android (Activity, boutons, champs de saisie, intents).

1. Structure minimale d'un fichier Kotlin

```
package com.example.monapplication

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

• package : indique le paquetage du fichier.

• import : permet d'utiliser une classe ou une fonction d'une bibliothèque.

• class : déclare une classe.

• : AppCompatActivity : indique que MainActivity hérite de AppCompatActivity.

• override : indique que l'on redéfinit une méthode héritée.

• onCreate() : méthode appelée lors de la création de l'activité.

• setContentView(...) : associe l'activité à son interface XML.

À retenir — En Kotlin, on n'écrit généralement pas de point-virgule ; à la fin des instructions.

2. Variables et constantes

```
var age: Int = 20          // variable modifiable
val nom: String = "Amine"  // référence non réaffectable
var moyenne = 14.5         // type Double déduit
val estInscrit = true      // type Boolean déduit
```

var permet de réaffecter une variable.
val empêche de réaffecter la variable ; c'est le choix à privilégier lorsque la valeur ne doit pas changer.

Types courants : Int (entier), Long (entier long), Double (réel), Float (réel simple précision), Boolean (vrai/faux), Char (caractère), String (chaîne).

```
var compteur = 0
compteur = compteur + 1

val prenom = "Sara"
println("Bonjour $prenom")
println("Dans 2 ans : ${age + 2}")
```

À retenir — L'interpolation $variable ou ${expression} insère une valeur dans une chaîne de caractères.

3. Commentaires et affichage

```
// Commentaire sur une seule ligne

/*
   Commentaire sur plusieurs lignes
*/

println("Affiche un texte avec retour à la ligne")
print("Affiche sans ajouter automatiquement une nouvelle ligne")
```

4. Opérateurs

Catégorie	Opérateurs	Exemple
Arithmétiques	+ - * / %	reste = 17 % 5
Comparaison	== != < > <= >=	age >= 18
Logiques	&& || !	age >= 18 && estInscrit
Affectation	= += -= *= /=	compteur += 1

5. Conditions

```
if (note >= 10) {
    println("Admis")
} else {
    println("Non admis")
}

// Plusieurs cas
if (note >= 16) {
    println("Très bien")
} else if (note >= 14) {
    println("Bien")
} else {
    println("Autre résultat")
}
```

Kotlin propose aussi when, pratique pour traiter plusieurs valeurs :

```
when (jour) {
    1 -> println("Lundi")
    2 -> println("Mardi")
    3 -> println("Mercredi")
    else -> println("Autre jour")
}
```

À retenir — Les conditions doivent être écrites entre parenthèses ; les blocs d'instructions sont délimités par { }.

6. Boucles

```
// Répéter un nombre de fois
for (i in 1..5) {
    println(i)       // 1, 2, 3, 4, 5
}

// De 0 à 4
for (i in 0 until 5) {
    println(i)
}

// Parcourir une collection
val noms = listOf("Ali", "Sarra", "Yasmine")
for (nom in noms) {
    println(nom)
}

// Répéter tant qu'une condition est vraie
var i = 1
while (i <= 5) {
    println(i)
    i++
}
```

• 1..5 inclut 5.

• 0 until 5 exclut 5.

• i++ augmente i de 1.

• break quitte la boucle ; continue passe à l'itération suivante.

7. Fonctions

```
fun afficherMessage(): Unit {
 println("Bienvenue")
 }
// ou
fun afficherMessage() {
    println("Bienvenue")
}

fun addition(a: Int, b: Int): Int {
    return a + b
}

fun carre(n: Int): Int = n * n

fun main() {
    afficherMessage()
    val resultat = addition(3, 4)
    println(resultat)
}
```

Dans fun addition(a: Int, b: Int): Int → a et b sont les paramètres et Int après les deux-points indique le type retourné.

En Kotlin, **Unit** est l'équivalent du void que l'on trouve dans des langages comme Java, C ou C++. Il indique qu'une fonction **ne retourne aucune valeur utile**.

À retenir — Les paramètres doivent avoir un type. Le type de retour peut parfois être déduit, mais il est utile de l'écrire pour apprendre.

8. Chaînes de caractères

```
val ville = "Monastir"
println(ville.length)           // longueur
println(ville.uppercase())      // MONASTIR
println(ville.lowercase())      // monastir
println(ville.startsWith("Mon"))

// Chaîne multilignes
val texte = """
    Première ligne
    Deuxième ligne
""".trimIndent()
```

Les chaînes sont immuables : les fonctions comme uppercase() renvoient une nouvelle chaîne, elles ne modifient pas la chaîne initiale.

9. Collections : List et MutableList

```
val couleurs = listOf("Rouge", "Vert", "Bleu") // lecture seule
println(couleurs[0])                           // Rouge
println(couleurs.size)                         // 3

val notes = mutableListOf(12, 15, 9)
notes.add(16)
notes[0] = 13
notes.remove(9)

for (note in notes) {
    println(note)
}
```

Les indices commencent à 0. Une List ne permet pas d'ajouter ou de remplacer des éléments via cette référence ; MutableList permet ces modifications.

10. Nullabilité : éviter les erreurs

```
var prenom: String = "Mouna"
// prenom = null // interdit

var telephone: String? = null  // le ? autorise null
telephone = "22123456"

println(telephone?.length)     // accès sûr
println(telephone ?: "Non renseigné") // valeur par défaut

if (telephone != null) {
    println(telephone.length)
}
```

• String : la valeur ne peut pas être null.

• String? : la valeur peut être null.

• ?. : exécute l'accès seulement si l'objet n'est pas null.

• ?: : fournit une valeur de remplacement.

À retenir — Éviter !! au début : cet opérateur force l'accès et peut provoquer une NullPointerException si la valeur est null.

11. Classes et objets : notions essentielles

```
class Etudiant(val nom: String, var note: Double) {
    fun estAdmis(): Boolean {
        return note >= 10
    }
}

val e = Etudiant("Amine", 14.5)
println(e.nom)
println(e.estAdmis())
```

Le constructeur principal est écrit entre parenthèses après le nom de la classe.

val nom crée une propriété non réaffectable ;

var note crée une propriété modifiable.

Un objet est créé en appelant le nom de la classe, sans le mot-clé new.

12. Kotlin dans une Activity Android avec XML Views

Exemple : l'interface activity_main.xml contient un TextView d'identifiant tvMessage, un EditText d'identifiant etNom et un Button d'identifiant btnValider.

```
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etNom = findViewById<EditText>(R.id.etNom)
        val tvMessage = findViewById<TextView>(R.id.tvMessage)
        val btnValider = findViewById<Button>(R.id.btnValider)

        btnValider.setOnClickListener {
            val nom = etNom.text.toString().trim()

            if (nom.isEmpty()) {
                Toast.makeText(this, "Saisir votre nom", Toast.LENGTH_SHORT).show()
            } else {
                tvMessage.text = "Bienvenue $nom"
            }
        }
    }
}
```

À comprendre :

- findViewById<Type>(R.id.identifiant) récupère une vue déclarée dans le XML.
- setOnClickListener { ... } définit les instructions exécutées au clic.
- etNom.text.toString() convertit le contenu éditable en String.
- trim() enlève les espaces au début et à la fin ; isEmpty() vérifie si la chaîne est vide.
- tvMessage.text = ... modifie le texte du TextView.
- Toast.makeText(...).show() affiche un petit message temporaire.

À retenir — Les identifiants R.id.etNom, R.id.tvMessage et R.id.btnValider doivent exister dans les fichiers XML. Les noms doivent correspondre exactement.

13. Lire un nombre saisi dans un EditText

```
val saisie = etNombre.text.toString()
val nombre = saisie.toIntOrNull()

if (nombre == null) {
    Toast.makeText(this, "Saisir un entier valide", Toast.LENGTH_SHORT).show()
} else {
    val double = nombre * 2
    tvResultat.text = "Résultat : $double"
}
```

toIntOrNull() renvoie un entier si la conversion réussit, sinon null. Cette méthode évite qu'une saisie non numérique provoque une erreur de conversion.

14. Intents explicites : premier exemple

Un Intent explicite permet de demander l'ouverture d'une activité précise de l'application.

```
import android.content.Intent


// Dans un listener de clic :
val intent = Intent(this, DeuxiemeActivity::class.java)
intent.putExtra("EXTRA_NOM", "Amine")
startActivity(intent)
```

Dans DeuxiemeActivity, on peut récupérer la donnée transmise :

```
val nom = intent.getStringExtra("EXTRA_NOM")
tvMessage.text = "Bonjour ${nom ?: "étudiant"}"
```

À retenir — DeuxiemeActivity doit être déclarée dans AndroidManifest.xml si elle n'est pas déjà enregistrée automatiquement par le modèle de projet.

15. Erreurs fréquentes à éviter

- Écrire = pour comparer : utiliser == pour tester l'égalité.
- Oublier les types dans les paramètres d'une fonction : fun somme(a: Int, b: Int).
- Confondre val et var : val ne permet pas de réaffecter la variable.
- Utiliser une valeur nullable sans contrôle : vérifier null, utiliser ?. ou ?:.
- Oublier .toString() pour convertir Editable en chaîne de caractères.
- Utiliser un identifiant R.id qui ne correspond pas à celui du XML.
- Lire une vue avant setContentView(...) : récupérer les vues après le chargement du layout.
- Convertir directement une saisie potentiellement incorrecte avec toInt() : préférer toIntOrNull().
- Oublier les accolades ou fermer incorrectement les parenthèses.

16. Tableau récapitulatif

Écriture Kotlin	Rôle
val x = 5	Déclarer une valeur non réaffectable
var x = 5	Déclarer une variable modifiable
fun f(a: Int): String	Déclarer une fonction avec paramètre et retour
if (...) { } else { }	Effectuer un choix
when (x) { ... }	Traiter plusieurs cas
for (i in 1..n)	Répéter sur un intervalle inclusif
while (condition)	Répéter tant que la condition est vraie
String?	Chaîne qui peut être null
val x = listOf(...)	Créer une liste en lecture seule
findViewById<Button>(R.id.btn)	Récupérer une vue XML
setOnClickListener { }	Réagir à un clic
Intent(this, Cible::class.java)	Créer un Intent explicite